

```

#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_TOKEN_LENGTH 100
#define MAX_HEADER_LENGTH 256

const char *keywords[] = {"auto",      "break",  "case",   "char",   "const",
                          "continue", "default", "do",     "double", "else",
                          "enum",     "extern", "float",  "for",    "goto",
                          "if",       "int",   "long",   "register", "return",
                          "short",    "signed", "sizeof", "static",  "struct",
                          "switch",   "typedef", "union",  "unsigned", "void",
                          "volatile", "while"};

const int num_keywords = sizeof(keywords) / sizeof(keywords[0]);

int is_keyword(const char *str) {
    for (int i = 0; i < num_keywords; i++) {
        if (strcmp(str, keywords[i]) == 0) {
            return 1;
        }
    }
    return 0;
}

int is_separator(char c) {
    return (c == '(' || c == ')' || c == '{' || c == '}' || c == '[' ||
            c == ']' || c == ',' || c == ';' || c == ':' || c == '.' || c == '#');
}

int is_delimiter(char c) { return (c == ';'); }

int is_operator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/' || c == '%' ||
            c == '=' || c == '<' || c == '>' || c == '&' || c == '|' ||
            c == '^' || c == '!' || c == '~');
}

int main() {
    FILE *file = fopen("lexAnalysis.txt", "r");

    if (file == NULL) {
        printf("Error opening file.\n");
        return 1;
    }

    char token[MAX_TOKEN_LENGTH];
    char header[MAX_HEADER_LENGTH];
    int c;
    int i = 0;

    while ((c = fgetc(file)) != EOF) {
        if (c == '#') {
            token[i++] = c;
            while ((c = fgetc(file)) != EOF && c != '\n') {
                token[i++] = c;
            }
            token[i] = '\0';
            i = 0;
            if (strncmp(token, "#include", 8) == 0) {
                char *start = strchr(token, '<');
                char *end = strchr(token, '>');
                if (start && end && start < end) {
                    strncpy(header, start + 1, end - start - 1);
                }
            }
        }
    }
}

```

```

        header[end - start - 1] = '\0';
        printf("Header File: #include<%s>\n", header);
    } else {
        start = strchr(token, '"');
        end = strrchr(token, '"');
        if (start && end && start < end) {
            strncpy(header, start + 1, end - start - 1);
            header[end - start - 1] = '\0';
            printf("Header File: %s\n", header);
        } else {
            printf("Preprocessor Directive: %s\n", token);
        }
    }
} else {
    printf("Preprocessor Directive: %s\n", token);
}
}

else if (isalpha(c) || c == '_') {
    token[i++] = c;

    while ((c = fgetc(file)) != EOF && (isalnum(c) || c == '_')) {
        token[i++] = c;
    }

    token[i] = '\0';

    if (is_keyword(token)) {
        printf("Keyword: %s\n", token);
        i = 0;
    } else {
        printf("Identifier: %s\n", token);
        i = 0;
        ungetc(c, file);
    }
}

else if (isdigit(c)) {
    token[i++] = c;
    while ((c = fgetc(file)) != EOF && (isdigit(c) || c == '.')) {
        token[i++] = c;
    }
    token[i] = '\0';
    printf("Number: %s\n", token);
    i = 0;
    ungetc(c, file);
}

else if (is_delimiter(c)) {
    printf("Delimiter: %c\n", c);
}

else if (is_separator(c)) {
    printf("Separator: %c\n", c);
}

else if (is_operator(c)) {
    printf("Operator: %c\n", c);
}

else if (c == '"') {
    token[i++] = c;
    while ((c = fgetc(file)) != EOF && c != '"') {
        token[i++] = c;
        if (c == '\\') {
            token[i++] = fgetc(file);

```

```
    }  
    }  
    token[i++] = c;  
    token[i] = '\\0';  
    printf("String: %s\\n", token);  
    i = 0;  
} else if (!isspace(c)) {  
    printf("Other: %c\\n", c);  
}  
}  
}  
fclose(file);  
return 0;  
}
```